

Chapter 10

Priority Queues and Heaps

How to read these notes

- ★ marks material that is core, high-yield, and worth memorizing.
- Content inside a gray **X** box is **not required for this class**. It is included for completeness only.

10.1 Introduction to Priority Queues

Why we need them

- Stacks and queues are **container adaptors** that access data by **order of insertion**:
 - Stack: last-in, first-out (LIFO).
 - Queue: first-in, first-out (FIFO).
- Some problems need access by a **priority value**, not insertion order:
 - Emergency call center**: dispatch calls by urgency, not by arrival order.
 - Stock exchange**: match the highest buy price with the lowest sell price. Priority is the price, not the time of entry.

Definition and interface

- ★ A **priority queue** is a data structure that lets you extract/pop the element with the **highest priority**, where priority is determined by a value that can be compared (e.g. with `operator<`).

Function	Behavior
<code>.push(val)</code>	Pushes element <code>val</code> into the priority queue
<code>.pop()</code>	Removes the element with the highest priority
<code>.top()</code>	Returns a reference to the element with the highest priority
<code>.size()</code>	Returns the number of elements
<code>.empty()</code>	Returns whether the priority queue is empty

- Example: push 1, 5, 2, 8, 4 into a max priority queue. `.pop()` removes 8, since it has the largest value.

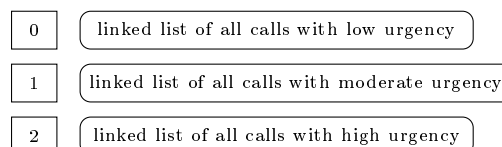
★ Summary of implementations

Implementation	<code>.push()</code>	<code>.pop()</code>
Array of linked lists (only for small integer priorities)	$\Theta(1)$	$\Theta(1)$
Unsorted sequence container	$\Theta(1)$	$\Theta(n)$
Sorted sequence container	$\Theta(n)$	$\Theta(1)$
Heaps	$\Theta(\log(n))$	$\Theta(\log(n))$

10.2 List and Sequence Container Implementations

10.2.1 Array of Linked Lists

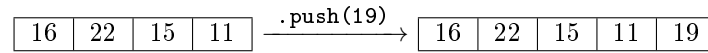
- Works only when data can take on a **small, fixed number of priority values**.
- Array size = number of possible priorities; each index holds a linked list of all elements with that priority.
- `.push()`: check the priority, append to the correct list $\Rightarrow \Theta(1)$.
- `.pop()`: remove from the linked list of the highest priority $\Rightarrow \Theta(1)$.



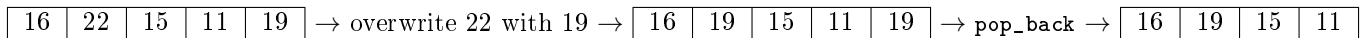
- Limitation**: fails when priorities can take on many/infinite values. A stock price could be \$5.00, \$5.01, \$5.02, ... ; you would need one linked list per possible price, which is impractical.

10.2.2 Unordered Sequence Container

- Stores elements in **no defined order**; usually a **vector** is the underlying container.
- `.push()`: append to the end $\Rightarrow \Theta(1)$.
- `.pop()`: linear search for the highest priority element $\Rightarrow$ worst-case $\Theta(n)$. Erasing from the middle of a vector shifts everything after it left $\Rightarrow$ also $\Theta(n)$ on average.

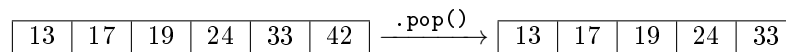
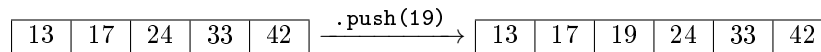


- ★ **Optimization**: because order does not matter, do not shift. **Overwrite** the element to remove with the **last** element, then `.pop_back()` (always constant time, unlike erasing at an arbitrary position).



10.2.3 Sorted Sequence Container

- The container must stay **sorted at all times**, so the highest priority element sits at the **back** (where vector removal is cheapest).
- `.push()`: find the insertion point with binary search (`std::lower_bound()` returns an iterator to the correct insertion point; covered in Chapter 11), but all elements after that point must shift $\Rightarrow \Theta(n)$.
- `.pop()`: `.pop_back()` the last element $\Rightarrow \Theta(1)$.



Is it better to have a fast `push` or a fast `pop`?

- It does not matter. Over the **full life cycle** of one element (inserted, then eventually removed), both implementations cost $\Theta(n)$ total. The only difference is **when** the $\Theta(n)$ operation happens.
- $\Theta(n)$ for either operation is still not ideal $\Rightarrow$ use a **heap**.

10.3 Binary Heaps

10.3.1 Tree and Heap Definitions

- ★ A **binary heap** implements a priority queue with worst-case $\Theta(\log(n))$ insertion and removal. The full life cycle of an element is $2 \times \Theta(\log(n)) = \Theta(\log(n))$, better than either sequence container.

Trees

- A **tree** is a graph (nodes connected by edges) that is **connected** and has **no cycles**. Any two nodes are connected by a single unique shortest path.
- **Parent**: the immediate ancestor (one level above a node).
- **Child**: an immediate descendant (one level below a node).
- **Ancestor**: any node closer to the root along a connected upward path.
- **Descendant**: any node farther from the root along a connected downward path. (Treat the node as the root of a subtree; the subtree holds all its descendants.)
- **Root**: the topmost node; the common ancestor of all nodes.
- **Leaf**: a node with no children.
- **Internal node**: a node with children (not a leaf).

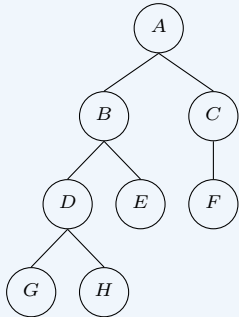
Measurements

- **Size** = number of nodes. Empty tree $\rightarrow 0$. Otherwise: $\text{size}(\text{left child}) + \text{size}(\text{right child}) + 1$.
- **Height** of a node = $\max \text{height of its children} + 1$; a leaf has height 1. (Equivalently: the max distance from that node down to a `nullptr`.)
- **Depth** of a node = $\text{depth of its parent} + 1$; the root has depth 1. Depth measures distance *to the root*; height measures distance *to the bottom*.

Binary trees

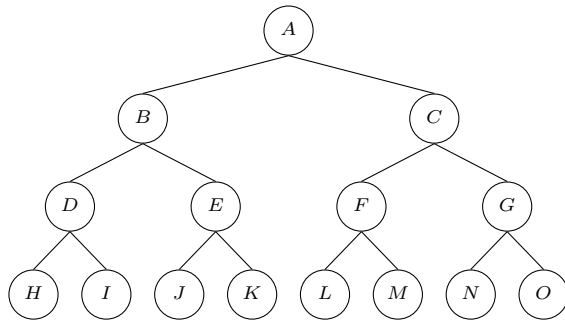
- **Binary tree:** every node has **at most two** children.
- **Proper / full binary tree:** every node has **0 or 2** children (no node has exactly one child).
- ★ **Complete binary tree:** every depth is completely filled **except possibly the last**, and the last row is filled **left to right with no gaps**.

Example: labeling a binary tree

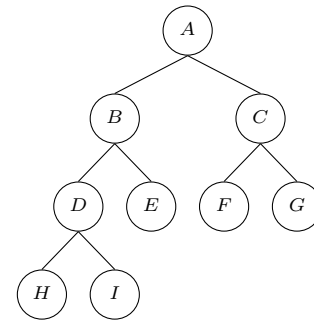


- A is the root. E, F, G, H are leaves. A, B, C, D are internal nodes.
- G and H are children of D; B is the parent of D and E.
- A and B are ancestors of D; D, E, G, H are descendants of B.
- Size of the tree = 8.
- Heights: E, F, G, H = 1; C and D = 2; B = 3; A = 4.
- Depths: A = 1; B, C = 2; D, E, F = 3; G, H = 4.

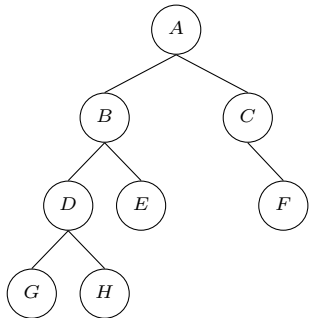
Completeness examples



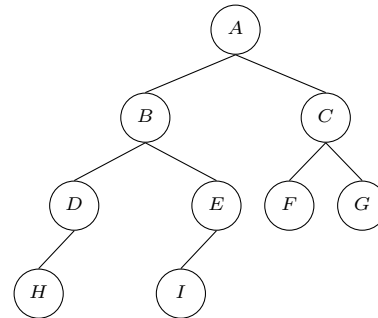
Complete: every depth is completely filled.



Complete: the lowest level is not full, but its nodes are as far left as possible.



Not complete: the second-to-last level has a gap (C has no left child). Only the final depth may be unfilled.



Not complete: all but the last level is full, but the last-level nodes are not packed left (D has no right child).

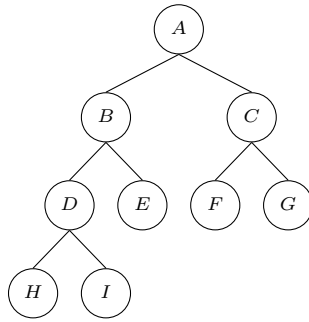
10.3.2 Representing a Binary Tree in Memory

Pointer-based

```
template <typename T>
struct Node {
    T data;           // data of node
    Node* left;       // pointer to left child
    Node* right;      // pointer to right child
};
```

- Efficient for moving down parent → child.
- Uses extra memory: every node stores two pointers on top of its data.
- ★ **Vector-based (works for complete trees)**
- Append the tree **row by row** (level order) into a vector.

- Completeness guarantees **no gaps** in the array. If a middle node were missing, its index would sit empty, wasting memory and making the vector larger than the node count.



	A	B	C	D	E	F	G	H	I
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Index 0 is left empty so that the root sits at index 1 (see below).

★ Index math (root stored at index 1)

Given a node at index i in a heap of size n :

- Parent: index $i/2$ (integer division). Parent of F (index 6) is C (index 3); parent of G (index 7) is C ($7/2 = 3$).
- Left child: index $2i$. Left child of B (index 2) is D (index 4).
- Right child: index $2i + 1$. Right child of B (index 2) is E (index 5).
- Leaf** if $i > n/2$. With $n = 9$: E (index 5) is a leaf since $5 > 4$.
- Internal** if $i \leq n/2$. With $n = 9$: D (index 4) is internal since $4 \leq 4$.

Remark: the dummy element at index 0

Storing a dummy at index 0 wastes space and makes invariants confusing (e.g. `pq.size() == pq.data.size() - 1`). A better option is private helpers that translate 1-indexing onto a 0-indexed vector. All remaining code in this chapter assumes a dummy at index 0 for simplicity.

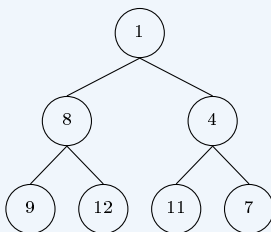
```
T& get_element(size_t idx) { return data[idx - 1]; }
const T& get_element(size_t idx) const { return data[idx - 1]; }
```

10.3.3 Heap Ordering

- A tree is **heap-ordered** if, for every node, each child has priority **less than or equal to** its parent. No node has priority greater than the root.
- Max-heap**: every node's value $\geq$ its children's values (larger = higher priority).
- Min-heap**: every node's value $\leq$ its children's values (smaller = higher priority).
- ★ **A binary heap must satisfy BOTH properties:**
 - The tree is **complete**.
 - The contents are **heap-ordered**.
- Max-heaps give easy access to the largest value; min-heaps give easy access to the smallest. That is why heaps back priority queues.

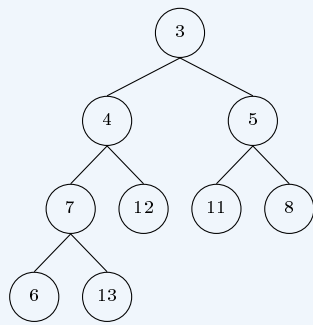
Example 10.1 — Which of these are min-heaps / max-heaps?

Strategy: draw the tree first. If the root is the smallest value, check the min-heap rule (no child smaller than its parent). If the root is the largest, check the max-heap rule (no child larger than its parent).



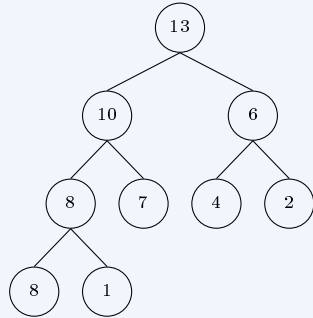
(a) [1, 8, 4, 9, 12, 11, 7]

Min-heap, since every parent is $\leq$ its children: $1 \leq 8, 4$; $8 \leq 9, 12$; $4 \leq 11, 7$.



(b) [3, 4, 5, 7, 12, 11, 8, 6, 13]

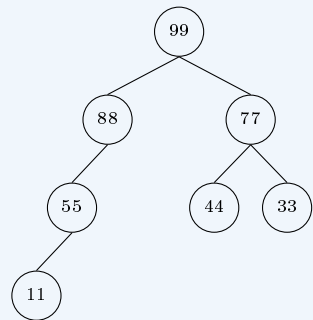
Neither. 7 has children 6 and 13, and $7 \not\leq 6$, so the min-heap ordering breaks (the rest of the elements do follow min-heap order).



(c) [13, 10, 6, 8, 7, 4, 2, 8, 1]

Max-heap, since every parent is $\geq$ its children: $13 \geq 10, 6$; $10 \geq 8, 7$; $6 \geq 4, 2$; $8 \geq 8, 1$ (ties are allowed).

Example 10.2 — Is this a valid max-heap?

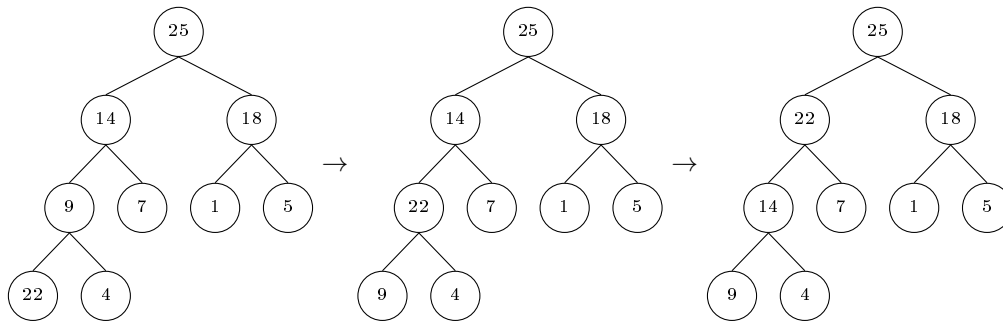


No. Even though this tree *is* heap-ordered (no child outranks its parent), it is **not complete**: there are gaps above the last level. Completeness is required for a binary heap.

10.3.4 Fix Up

- Used when the priority of an element is **increased** (it needs to move **up**).
- ★ Repeatedly swap the modified element with its **parent** until either:
 - you reach the root, or
 - you reach a parent with **higher or equal** priority.

Max-heap, node 2 is updated to 22. It swaps with 9, then with 14, then stops (its parent 25 is larger).



```
// this function returns whether val1 has a lower priority than val2
template <typename T>
bool comp_priority_less(const T& val1, const T& val2);

template <typename T>
void fix_up(std::vector<T>& data, size_t index) {
```

```

// swap modified element with parent until root is reached or a parent
// with a greater than or equal priority is found
while (index > 1 && comp_priority_less(data[index / 2], data[index])) {
    swap(data[index], data[index / 2]);
    index /= 2;
} // while
} // fix_up()

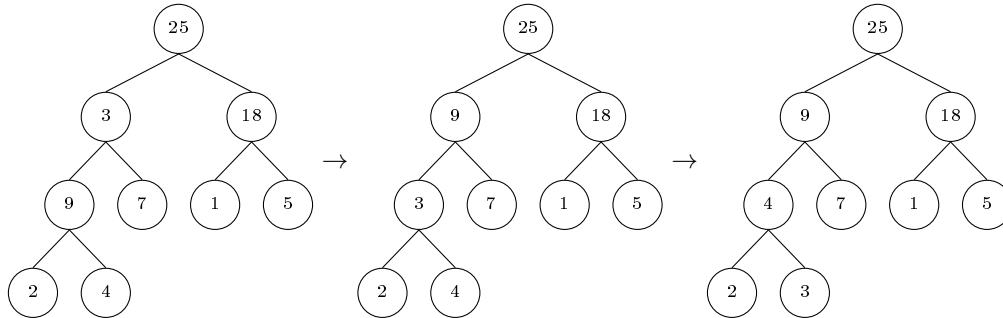
```

- **Complexity:** worst case, the element travels from the bottom level to the root; number of swaps = number of levels $\approx \log(n) \Rightarrow \Theta(\log(n))$.
- **Why $\Theta(\log(n))$ levels and not $\Theta(n)$?** Completeness. A complete tree with n nodes must have $\Theta(\log(n))$ levels because every level except the last is full. A stick-like tree with $\Theta(n)$ levels is not complete.

10.3.5 Fix Down

- Used when the priority of an element is **decreased** (it needs to move **down**).
- ★ Repeatedly swap the modified element with its **highest priority child** until either:
 - you reach the bottom of the heap, or
 - you reach a position where no child has higher priority.

Max-heap, node 14 is updated to 3. It swaps with its largest child 9, then with 4, then stops.



```

template <typename T>
void fix_down(std::vector<T>& data, size_t index) {
    size_t heap_size = data.size() - 1;
    while (2 * index <= heap_size) {
        // initialize highest priority child to left child
        size_t larger_child = 2 * index;
        // if right child has higher priority, set larger child to right child
        if (larger_child < heap_size &&
            comp_priority_less(data[larger_child], data[larger_child + 1])) {
            ++larger_child;
        } // if
        // if children all have lower priority, the heap is restored, so break
        if (comp_priority_less(data[larger_child], data[index])) {
            break;
        } // if
        // otherwise, swap the modified element with the largest child
        std::swap(data[index], data[larger_child]);
        index = larger_child; // set new index for next iteration
    } // while
} // fix_down()

```

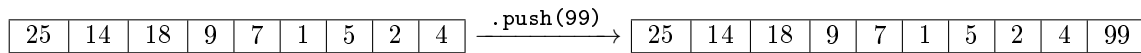
- Slightly more complex than fix up because it must check whether a left/right child actually exists.
- **Complexity:** at most $\Theta(\log(n))$ swaps (one per level) $\Rightarrow$ worst case $\Theta(\log(n))$.

10.3.6 Inserting and Removing Elements

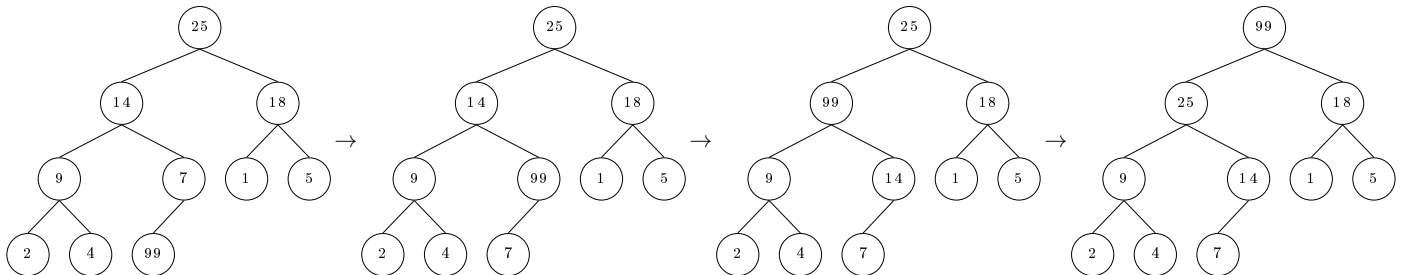
★ Push

1. `.push_back()` the element onto the underlying vector (this is the next open position on the bottom level, filling left to right).
 2. Call **fix up** on it to swap it up to the correct position.
- A heap's vector is not sorted, so binary search cannot find an insertion point; that is why we push to the back first.

```
template <typename T>
void push(std::vector<T>& data, const T& val) {
    data.push_back(val);
    fix_up(data, data.size());
} // push()
```



99 lands in the next open spot (as a child of 7), then fixes up past 7, 14, and 25.



- **Complexity:** $\Theta(1)$ push back + $\Theta(\log(n))$ fix up = $\Theta(1 + \log(n)) = \Theta(\log(n))$.

Why naive removal fails

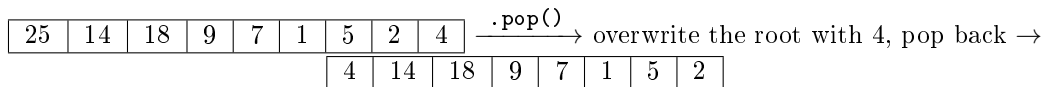
```
template <typename T>
void pop(std::vector<T>& data) {
    data.erase(data.begin() + 1); // NOTE: this implementation of pop is incorrect!
} // pop()
```

- Erasing index 1 shifts every remaining element left $\Rightarrow \Theta(n)$, which defeats the purpose.
- Worse, the result is **not a valid heap**. Popping 25 from [25, 14, 18, 9, 7, 1, 5, 2, 4] and shifting gives [14, 18, 9, 7, 1, 5, 2, 4]: now 14 is at the root even though 18 has higher priority.

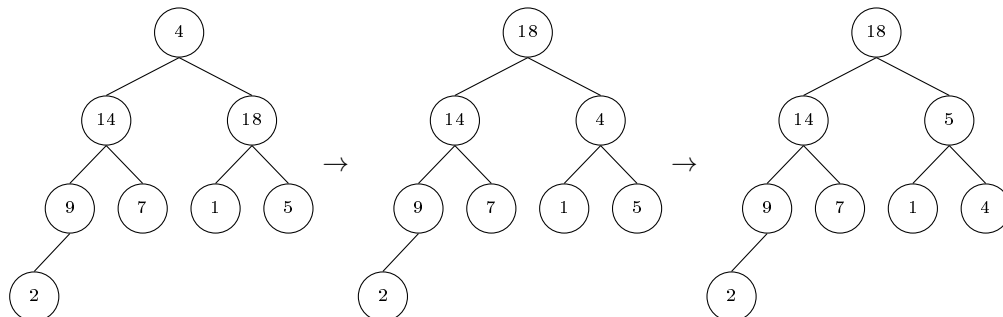
★ Pop (correct approach)

1. Overwrite the **root** with the element at the **back** of the vector.
 2. `.pop_back()` the last element.
 3. Call **fix down** on the new root.
- This works because only one element (the one moved to the root) changed position; everything else is still heap-ordered.

```
template <typename T>
void pop(std::vector<T>& data) {
    data[1] = data.back();
    data.pop_back();
    fix_down(data, 1);
} // pop()
```



4 is now at the root, so fix it down: swap with 18, then with 5.



- **Complexity:** $\Theta(1)$ overwrite + $\Theta(1)$ pop back + $\Theta(\log(n))$ fix down = $\Theta(\log(n))$.

Example 10.3 — Insert 55 into this max-heap

Heap: root 57, children 54 and 52; 54's children 45 and 49; 52's children 48 and 42; 45's children 41 and 43.

- 55 is added at the very back (as the left child of 49), then fixed up.
- 55 swaps with 49, then swaps with 54.
- Final array: [57, 55, 52, 45, 54, 48, 42, 41, 43, 49]

The array representation is simply the elements in **level order** (left to right across each level).

Example 10.4 — Pop 57 off the same max-heap

- Overwrite 57 with the last element (43), pop the back, then fix down.
- 43 swaps with its largest child 54, then swaps with 49.
- Final array: [54, 49, 52, 45, 43, 48, 42, 41]

Example 10.5 — Push 32, 52, 11, 23, 59, 10 into an empty min-heap

1. **32**: trivial (heap was empty).
2. **52**: goes as the left child of 32. Lower priority than 32, so no fix needed.
3. **11**: goes as the right child of 32. Higher priority $\Rightarrow$ fix up: 11 swaps with 32. Heap: [11, 52, 32].
4. **23**: goes as the left child of 52. Higher priority $\Rightarrow$ swaps with 52. Heap: [11, 23, 32, 52].
5. **59**: goes as the right child of 23. Lower priority, no swap.
6. **10**: goes as the left child of 32. Higher priority $\Rightarrow$ swaps with 32, then with 11.

Final array: [10, 23, 11, 52, 59, 32]

10.4 Heapify

10.4.1 Turning an Arbitrary Container into a Heap

- Naive approach: push all values into a heap one at a time.
 - Time: $\Theta(n \log(n))$ (n pushes, each $\Theta(\log(n))$).
 - Space: extra $\Theta(n)$ memory, since you copy into a new container.
- ★ **Heapify** does it in $\Theta(n)$ time and $\Theta(1)$ auxiliary space (in place).
- Idea: if fix up / fix down can repair any single element, then iterating over the container and calling fix up or fix down on each element should build a heap.

Four candidate approaches

- **Idea #1**: forward (top $\rightarrow$ bottom), call **fix up** on each element. **Works**
- **Idea #2**: forward (top $\rightarrow$ bottom), call **fix down** on each element. **Fails**
- **Idea #3**: backward (bottom $\rightarrow$ top), call **fix up** on each element. **Fails**
- **Idea #4**: backward (bottom $\rightarrow$ top), call **fix down** on each element. **Works**
- Forward iteration = start at index 1 (the root) and move to the end. Backward iteration = start at the back (rightmost leaf on the lowest level) and move to the front.
- ★ **Rule: you must fix in the direction you came from.**
 - Traversing top $\rightarrow$ bottom $\Rightarrow$ fix **up**.
 - Traversing bottom $\rightarrow$ top $\Rightarrow$ fix **down**.

Analogy (footnote in the text)

Think of walking through fresh snow. You leave footprints *behind* you while the snow ahead stays untouched. To clean up, you fix what is behind you. Same with heapify: fix in the direction opposite to your movement, since that is where the mess can occur.

Proof that Idea #2 fails (forward + fix down)

Build a max-heap from [1, 2, 3, 4, 5, 6, 7].

- Fix down index 1 (value 1): 1 swaps with 3, then with 7. Tree becomes root 3, children 2 and 7 (7's children 6 and 1).

- Fix down index 2 (value 2): 2 swaps with 5.
- Fix down index 3 (value 7): already larger than its children, no change. The remaining elements (4, 2, 6, 1) are leaves, so nothing happens.

Final tree: root **3**, children 5 and 7. **Not a valid max-heap**, since $3 < 5$ and $3 < 7$.

Why: the smallest element (1) was swapped with 3, forcing 3 to the top. Since we iterate forward while only fixing downward, the 3 is never revisited and gets stuck at the root.

Proof that Idea #3 fails (backward + fix up)

Build a max-heap from [5, 3, 6, 1, 2, 4, 7].

- Fix up the last element (7): 7 swaps with 6, then with 5.
- Fix up 4, 2, 1, 5, 3, 7 in that order: none of these move, since each is smaller than its parent.

Final tree: root **7**, children 3 and 5, where 5 has children 4 and **6**. **Not a valid max-heap**, since $6 > 5$.

Why: the largest element (7) was swapped with 6, forcing 6 down. Since we move toward the top while only fixing upward, the 6 is never revisited and gets stuck at the bottom.

10.4.2 ★ Top-Down vs. Bottom-Up Heapify

- **Top-Down Heapify:** traverse root $\rightarrow$ leaves, calling **fix up** on each element. Worst case $\Theta(n \log(n))$.
– n calls to fix up, each worst-case $\Theta(\log(n))$.
- **Bottom-Up Heapify:** traverse leaves $\rightarrow$ root, calling **fix down** on each element. Worst case $\Theta(n)$.

Why bottom-up is $\Theta(n)$

- Not every fix call does the same amount of work.
 - **Fix up** is most expensive at the **bottom** (up to $\Theta(\log(n))$ swaps) and cheapest at the top (fix up on the root is $\Theta(1)$, it cannot move higher).
 - **Fix down** is most expensive at the **top** (up to $\Theta(\log(n))$ swaps) and cheapest at the bottom (fix down on a leaf is $\Theta(1)$, it has no children).
- There are far more nodes near the bottom: in a binary heap of n nodes, **at least half are leaves**.
- So half of the nodes are the expensive case for fix up and the free case for fix down. Bottom-up heapify effectively **skips half the tree**.

Number of Nodes	Max Levels to Fix (distance to leaf)	Max Swaps Needed
$n/2$	0	$n/2 \times 0 = 0$
$n/4$	1	$n/4 \times 1 = n/4$
$n/8$	2	$n/8 \times 2 = 2n/8$
$n/16$	3	$n/16 \times 3 = 3n/16$
$n/32$	4	$n/32 \times 4 = 4n/32$
...	...	...
1 (the root)	$\log(n)$	$\log(n)$

- Total work = $n \left(0 + \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots \right)$, and the series in parentheses **converges to a constant**, so the total is $\Theta(n)$.
- (The proof of that convergence is outside the scope of the class; you only need to know the result is $\Theta(n)$.)

Example 10.6 — Bottom-up heapify [54, 66, 23, 74, 62, 33, 19] into a min-heap

Tree: root 54, children 66 and 23; 66's children 74 and 62; 23's children 33 and 19.

- Start at the back and move forward, calling fix down. Fix down does nothing on leaves, so **skip the entire bottom row**.
- Start at the last node with a child (index 3, value **23**): 23 swaps with 19.
- Index 2, value **66**: 66 swaps with 62.
- Index 1 (root), value **54**: 54 swaps with 19, then swaps with 23.

Final array: [19, 62, 23, 74, 66, 33, 54]

✗ Not required: STL heapify (`std::make_heap`, `std::push_heap`, `std::pop_heap`)

You do **not** need to know these for this class, and you should **not** use them on Project 2. Still handy to know they exist for future classes.

`std::make_heap(first, last, comp)` — rearranges the range `[first, last)` into a heap. The comparator is optional; without it a **max-heap** is created. Priority rule matches `std::priority_queue`: `comp(a, b)` returns `true` if `a` has *lower* priority than `b`. So `std::less<>` gives a max-heap and `std::greater<>` gives a min-heap.

```
std::vector<int32_t> v1 = {4, 6, 2, 9, 7, 3, 8, 1, 5};
std::make_heap(v1.begin(), v1.end()); // max-heap: 9 7 8 6 4 3 2 1 5

std::vector<int32_t> v2 = {4, 6, 2, 9, 7, 3, 8, 1, 5};
std::make_heap(v2.begin(), v2.end(), std::greater<int>()); // min-heap: 1 4 2 5 7 3 8 9 6
```

`std::push_heap(first, last, comp)` — inserts the value at `last - 1` into the heap defined by `[first, last - 1)` (i.e. fixes the back value up).

```
std::vector<int32_t> v3 = {4, 6, 2, 9, 7, 3, 8, 1, 5};
std::make_heap(v3.begin(), v3.end()); // 9 7 8 6 4 3 2 1 5
v3.push_back(10); // 9 7 8 6 4 3 2 1 5 10
std::push_heap(v3.begin(), v3.end()); // 10 9 8 6 7 3 2 1 5 4
```

`std::pop_heap(first, last, comp)` — swaps the value at `first` with the one at `last - 1` and re-heapifies `[first, last - 1)` (i.e. fixes the new front down). It does **not** remove anything; call `.pop_back()` yourself.

```
std::vector<int32_t> v4 = {4, 6, 2, 9, 7, 3, 8, 1, 5};
std::make_heap(v4.begin(), v4.end()); // 9 7 8 6 4 3 2 1 5
std::pop_heap(v4.begin(), v4.end()); // 8 7 5 6 4 3 2 1 9
v4.pop_back(); // actually removes 9: 8 7 5 6 4 3 2 1
```

Complexities: `make_heap` is $\Theta(n)$ (it heapifies). The other two assume the range is already a valid heap and only fix one value, so both are $\Theta(\log(n))$.

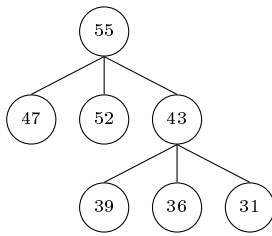
10.4.3 ★ Summary of Binary Heaps

- A heap implements a priority queue with $\Theta(\log(n))$ push and pop, better than sequence container implementations.
- The most common heap is the **binary heap** (at most two children per node). It is visualized as a tree but usually **stored in a vector**.
- To be a heap the tree must be **complete** and **heap-ordered**.
- Two core operations, both $\Theta(\log(n))$:
 - **fix up**: swap with the parent until in the correct position.
 - **fix down**: swap with the highest-priority child until in the correct position.
- **Push**: add to the back of the vector, then fix up.
- **Pop**: overwrite the front with the back element, pop back, then fix down on the new front.
- **Build a heap**: bottom-up heapify. Start at the last internal node, iterate to the front, calling fix down. This beats top-down because fix down is cheap for leaves, and most nodes are near the leaves.

10.5 Pairing Heaps

10.5.1 Structure

- Like binary heaps, no node's priority can exceed its parent's. Unlike binary heaps:
 - a node can have **more than two children**;
 - the tree does **not** need to be complete.
- Because of that, array-based storage is not feasible (no easy index math for many children) $\Rightarrow$ **pointer-based**.
- ★ Instead of storing a pointer to every child, each node stores:
 - a **child** pointer, to its **leftmost child** only;
 - a **sibling** pointer, linking all nodes that share a common parent.
- This is for efficiency: a node with 281 children should not track 281 pointers.
- **Max pairing heap** gives fast access to the largest value; **min pairing heap** to the smallest.



Example max pairing heap. The root 55 has three children but only stores 47 as its **child**. The rest are chained by **sibling**: 47's sibling is 52, and 52's sibling is 43.

The fourth pointer: parent OR previous

- Each node needs **one** extra pointer, either **parent** or **previous**. Only one is needed for full functionality with optimal complexities. Having both speeds things up slightly, but the memory overhead per node is not worth it.
- parent** pointer: points to the node's parent.
- previous** pointer: points to the sibling directly to its **left**; if the node is already the leftmost sibling, **previous** points to its **parent**.

```

template <typename T>
class Node {
    T data;
    Node* child;
    Node* sibling;
    Node* parent;
public:
    explicit Node(const T& val)
        : data{val}, child{nullptr},
          sibling{nullptr},
          parent{nullptr} {}
};

template <typename T>
class Node {
    T data;
    Node* child;
    Node* sibling;
    Node* previous;
public:
    explicit Node(const T& val)
        : data{val}, child{nullptr},
          sibling{nullptr},
          previous{nullptr} {}
};
  
```

10.5.2 ★ Meld

- meld()** is the fundamental pairing heap operation: it combines two pairing heaps into one.
- Compare the two **roots**. The tree with the **lower priority root** becomes the **leftmost child** of the other tree. Ties are broken arbitrarily.
- Complexity:** $\Theta(1)$ — only a constant number of pointers move, regardless of heap size.

Pointers to adjust — parent pointer version

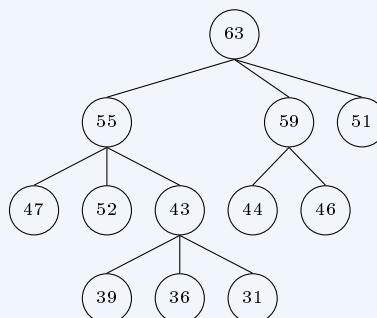
- sibling of the lower priority root → the leftmost child of the higher priority root.
- parent of the lower priority root → the higher priority root.
- child of the higher priority root → the lower priority root.

Pointers to adjust — previous pointer version

- sibling of the lower priority root → the leftmost child of the higher priority root.
- previous of the higher priority root's leftmost child → the lower priority root.
- child of the higher priority root → the lower priority root.
- previous of the lower priority root → the higher priority root.

Meld example

treeA has root 55; **treeB** has root 63. Since $55 < 63$, **treeA** is attached as the leftmost child of **treeB**:



10.5.3 Inserting and Removing Elements

Insert — $\Theta(1)$

- Simply **meld** the new single-node heap with the existing heap.
 - New element has lower priority than the root $\Rightarrow$ it becomes the leftmost child.
 - New element has higher priority than the root $\Rightarrow$ the entire existing heap becomes the leftmost child of the new element.
- Example (max pairing heap with root 18): `.push(17)` $\Rightarrow 17 < 18$, so 17 becomes the leftmost child of 18. `.push(19)` $\Rightarrow 19 > 18$, so the whole heap becomes the leftmost child of 19.

Remove (pop) — **amortized** $O(\log(n))$

- Removing the root leaves **multiple separate children**. Key insight: **each child is itself a valid pairing heap**, so rebuild by melding them back together.
- Two efficient approaches: **multi-pass** and **two-pass**.

Multi-pass approach

1. Break all existing sibling connections among the children.
2. Push all the children into a **queue** (done at the same time as step 1).
3. Take the two heaps at the **front** of the queue, meld them, and push the result to the **back**.
4. Repeat step 3 until only one pairing heap remains.

Two-pass approach

1. Break all existing sibling connections among the children.
2. Initialize a **queue** and a **stack** (or two deques). Push all children into the queue.
3. **Pass 1**: make a left-to-right pass over the trees in the queue, melding them **in pairs**, pushing each resulting tree onto the stack. If there is an odd number of children, the last tree goes on the stack unmelded.
4. **Pass 2**: start with the rightmost tree (top of the stack) and meld the remaining trees into it one at a time, popping from the stack until it is empty.

Remark: why multiple passes?

Why not just list the children and meld them one at a time into a single accumulated heap? That single-pass approach is not as efficient over repeated operations. Melding *pairs* on each pop produces better-shaped trees with fewer immediate children of the root over time, which is what gives pop its amortized $O(\log(n))$ bound. A single accumulating pass can leave the root with many immediate children, making future pops more expensive (more children to meld). Amortization is covered in Chapter 12.

10.5.4 Copy Constructor

- A pairing heap uses dynamic memory, so it needs a custom **copy constructor**, **destructor**, and **assignment operator**. The assignment operator can use **copy-swap** (copy the other heap into a temporary, then swap contents).
- You cannot just copy the root pointer; you must walk the whole tree and copy every node.
- ★ The copied heap does **not** need the same structure as the original. As long as it is a valid pairing heap holding the same elements, it is correct.

Process (queue-based, same idea as the queue-based search from the previous chapter):

1. Push the root node of the original heap into a queue.
 2. Retrieve the node at the front of the queue and pop it off.
 3. Push all of that node's **direct connections** into the queue: its child (if any) and its sibling (if any).
 4. Push the value of that node into the new pairing heap being constructed.
 5. Repeat steps 2–4 until the queue is empty.
- **Complexity:** $\Theta(n)$ (every node is visited and copied once).

Copy constructor walkthrough

Original heap: root 19, whose child is 18; 18's child is 17 and 18's sibling is 15; 17's sibling is 16.

- Take 19: push its child 18 into the queue; push 19 into the new heap. Queue: [18]
- Take 18: push child 17 and sibling 15; push 18 into the heap. Queue: [17, 15]
- Take 17: no child, push sibling 16; push 17 into the heap. Queue: [15, 16]

- Take 15: no connections; push 15 into the heap. Queue: [16]
- Take 16: no connections; push 16 into the heap. Queue: []

Queue is empty, so all elements are copied. The new heap has a different internal structure than the original, which is fine.

10.5.5 Destructor

- Identical to the copy constructor, except each node is **deleted** after it is taken out of the queue instead of being copied into a new heap.
1. Push the root node into a queue.
 2. Retrieve the element at the front of the queue and pop it off.
 3. Push its direct connections (**child** and **sibling**, if they exist) into the queue.
 4. **Delete** the node that was taken out.
 5. Repeat steps 2–4 until the queue is empty.
- **Complexity:** $\Theta(n)$ where n is the initial heap size.
 - Note the ordering: you must push a node's connections into the queue **before** freeing its memory.

10.5.6 Updating Priorities: Rebuilding an Invalid Heap

- If the data is scrambled and you do not know which elements are invalid, the best approach is to **break the whole heap apart and rebuild it**.
 - Again use a queue so each node is visited once. First push the root of the invalid heap into the queue and **set the current heap's root to nullptr**. Then repeat until the queue is empty:
1. Retrieve the node at the front of the queue and pop it off. This node is the root of a subheap and may still have connections that need breaking (this differs from the copy constructor / destructor, since here we rearrange an *existing* heap).
 2. Push its direct connections (**child** and **sibling**, if any) into the queue.
 3. Break all of the subheap's connections by setting **child**, **sibling**, and **parent/previous** to **nullptr**.
 4. **Meld** the removed single node with the current pairing heap.
- **Complexity:** $\Theta(n)$.
 - The rebuilt heap points to the same data as before; only the relationships between elements change.

Rebuild walkthrough

Invalid heap: root 27 with child 14; 14's sibling is 39; 39's child is 21 and 39's sibling is 23; 21's sibling is 45.

- Take 27, push child 14, break connections, meld with **nullptr** $\Rightarrow$ heap = {27}. Queue: [14]
- Take 14, push sibling 39, break, meld $\Rightarrow$ 14 has lower priority than 27, so 14 attaches under 27. Queue: [39]
- Take 39, push child 21 and sibling 23, break, meld $\Rightarrow$ 39 becomes the new root. Queue: [21, 23]
- Take 21, push sibling 45, break, meld. Queue: [23, 45]
- Take 23, no connections, break, meld. Queue: [45]
- Take 45, no connections, break, meld $\Rightarrow$ 45 becomes the new root. Queue empty.

Final heap: root 45 with 39 beneath it, and the other nodes melded in as children.

10.5.7 Increasing the Priority of a Single Element

- If exactly one node's priority is **increased** and you are handed that node, tearing down the whole heap is wasteful — the rest is still valid.
- ★ Just compare the updated node with its **parent**:
 - Still lower priority than its parent $\Rightarrow$ **do nothing**, the heap is still valid.
 - Higher priority than its parent $\Rightarrow$ **break the subtree off** from its siblings and **remeld** it with the root.
- The modified node's **children do not need to be detached**: since only the priority increased, they are guaranteed to still be in a valid position.
- (If the priority had been *decreased* — not covered in this class — the children would also need to be detached and remelded.)

Example: 38 is updated to 83

Original max pairing heap: root 79 with children 38 and 26; 38's children are 54 and 62; 26's children are 22 and 28.

- After the update, 83 is larger than its parent 79.
- The entire subtree rooted at 83 (including its children 54 and 62) is broken off and remelded with the original root.
- Result: 83 is the new root, with 79 (and its remaining subtree) melded underneath it.

Parent pointer vs. previous pointer trade-off

	parent pointer	previous pointer
Comparing to parent	Fast (direct access)	Slow (traverse leftward through siblings to find the parent)
Breaking sibling links	Slow (traverse rightward through siblings to find the node to the left)	Fast (direct access to the left sibling)

Remark

With a **previous** pointer, it may be worth skipping the parent check entirely and always breaking and remelding on every update: the parent is not easily accessible, and the cost of an unnecessary break-and-meld can beat iterating over all siblings to find the parent. With a **parent** pointer, it is still worth checking first, since the check is direct and can save unnecessary work.

10.6 The STL Priority Queue Container

- `#include <queue>` and declare:

```
std::priority_queue<TYPE, CONTAINER, COMPARATOR> name_of_pq;
```

- Like `std::stack<>` and `std::queue<>`, it is a **container adaptor** built on top of another container that actually stores the data.
- ★ The underlying **CONTAINER** must be **sequential and support random access**, because binary heaps rely on index math to find children. Valid: `std::vector<>`, `std::deque<>`. **Not** valid: `std::list<>`.
- ★ **COMPARATOR** rule (same as `std::make_heap()`): for any two elements `a` and `b`, if `comp(a, b)` returns `true`, then `a` has **lower priority** than `b`.

Defaults

- Only **TYPE** is required.
- Default **CONTAINER**: `std::vector<>`. Default **COMPARATOR**: `std::less<>` $\Rightarrow$ a **max** priority queue.

```
std::priority_queue<int32_t> max_pq;    // max PQ of ints, backed by a vector

// COMPARATOR must be the THIRD term, so the container must be spelled out too
std::priority_queue<int32_t, std::vector<int32_t>, std::greater<int32_t>> min_pq;

std::priority_queue<int, std::greater<int>> min_pq;    // does not compile!
```

Operations

Function	Behavior
<code>.push(val)</code>	Adds <code>val</code> to the priority queue
<code>.emplace(args)</code>	Constructs an element in place (no copies or moves) from constructor arguments
<code>.pop()</code>	Removes the highest priority element
<code>.top()</code>	Returns a const reference to the highest priority element
<code>.size()</code>	Returns the number of elements
<code>.empty()</code>	Checks if the priority queue is empty

- `.top()` is **const** because you may not modify an element while it is in the priority queue — `std::priority_queue<>` does **not** repair itself if a priority is invalidated.

- To change the top element safely: **pop it and re-push it** with the new value. For larger structs/classes you can mark members `mutable`, but be careful not to invalidate the ordering.

★ Range constructor

```
std::vector<int32_t> vec = {12, 66, 34, 25, 84, 25, 17, 98, 53};
std::priority_queue<int32_t> pq(vec.begin(), vec.end());
```

- This **heapifies** the range in $\Theta(n)$ time.
- Much better than pushing elements one at a time, which is worst-case $\Theta(n \log(n))$.

Footnote: why a binary heap and not a pairing heap?

Even though pairing heaps are theoretically more efficient, binary heaps often outperform more complicated heaps in practice because of **caching**: memory that is close together can be accessed sequentially faster. Also, `std::priority_queue<>` is **implementation defined** — it is not guaranteed to use a binary heap, only to conform to the standard's interface (though most libraries use some form of binary heap).

Example 10.7 — Custom comparator

```
struct Comp {
    bool operator() (const int32_t lhs, const int32_t rhs) const {
        return abs(lhs - 100) > abs(rhs - 100);
    } // operator()()
};

int main() {
    std::vector<int32_t> vec = {112, 166, 314, 251, 184, 25, 117, 98, 153};
    std::priority_queue<int32_t, std::vector<int32_t>, Comp> pq(vec.begin(), vec.end());
    std::cout << pq.top() << '\n';
} // main()
```

- `comp(112, 166)` returns `false` since `abs(112-100)` is not greater than `abs(166-100)`, so 112 does **not** have lower priority than 166.
- `comp(25, 166)` returns `true` since `abs(25-100) > abs(166-100)`, so 25 has lower priority than 166.
- So priority is determined by **distance from 100**: farther from 100 = lower priority.
- `.top()` prints the value closest to 100 $\Rightarrow$ **98**.

Example 10.8 — Multi-level comparator (stock orders)

```
struct StockOrderInfo {
    int32_t price; int32_t quantity; int64_t timestamp; int64_t order_id;
};
```

Execution priority for resting BUY orders:

1. Highest **price** first, regardless of quantity or time.
2. Tie on price $\Rightarrow$ earlier **timestamp** first.
3. Tie on price and time $\Rightarrow$ larger **quantity** first.
4. Tie on all three $\Rightarrow$ smaller **order_id** first (IDs are unique).

The comparator returns `true` when `lhs` has **lower** priority, so flip each rule: larger ID, smaller quantity, later timestamp, lower price $\Rightarrow$ lower priority.

```
struct StockOrderComparator {
    bool operator() (const StockOrderInfo& lhs, const StockOrderInfo& rhs) const {
        if (lhs.price == rhs.price) {
            if (lhs.timestamp == rhs.timestamp) {
                if (lhs.quantity == rhs.quantity) {
                    return lhs.order_id > rhs.order_id;
                } // if
                return lhs.quantity < rhs.quantity;
            } // if
            return lhs.timestamp > rhs.timestamp;
        } // if
        return lhs.price < rhs.price;
    } // operator()()
};
```

```
std::priority_queue<StockOrderInfo, std::vector<StockOrderInfo>,
                    StockOrderComparator> stock_pq;
```

Pushing orders 1023 (p50,q30,t12343), 1024 (p50,q35,t12343), 1025 (p50,q45,t12344), 1026 (p50,q45,t12344), 1027 (p60,q25,t12345) and popping until empty prints:

The orders are executed in the following order: 1027 1024 1023 1025 1026

10.7 Solving Problems Using Priority Queues

10.7.1 ★ k Largest Elements in an Array

Given n elements, print the k largest (in any order).

- **Solution 1 (naive)**: sort and take the last $k \Rightarrow \Theta(n \log(n))$.
- **Solution 2**: heapify into a **max-heap** ($\Theta(n)$) and pop k times ($\Theta(k \log(n)) \Rightarrow \Theta(n + k \log(n))$).
- **Solution 3 (best)**: use a **min-heap of size k** .
 1. Add the first k elements to the min-heap.
 2. For each of the remaining $n - k$ elements, compare with the **top** of the min-heap. If the new element is larger, **pop** the top and **push** the new value.
 - The min-heap always holds the k highest priority elements seen so far.
 - Time: $\Theta(n \log(k))$ (n pushes/pops, each $\Theta(\log(k))$). Auxiliary space: $\Theta(k)$.
 - Can be improved to $\Theta(k + (n - k) \log(k))$ by **heapifying** the first k elements instead of pushing them one at a time.

Walkthrough: 3 largest in [15, 61, 36, 24, 46, 47, 53, 28]

Push the first 3 into a min-heap: {15, 61, 36} (top = 15).

- $24 > 15 \Rightarrow$ pop 15, push 24. Heap: {24, 61, 36}, top = 24.
- $46 > 24 \Rightarrow$ pop 24, push 46. Heap: {36, 61, 46}, top = 36.
- $47 > 36 \Rightarrow$ pop 36, push 47. Heap: {46, 61, 47}, top = 46.
- $53 > 46 \Rightarrow$ pop 46, push 53. Heap: {47, 61, 53}, top = 47.
- $28 < 47 \Rightarrow$ not pushed.

Answer: **47, 61, 53**.

10.7.2 ★ Streaming Median

- A **stream** must be accessed sequentially; you cannot know how many elements are coming or what values are still ahead.
- **Naive**: store everything in a vector and sort whenever the median is needed $\Rightarrow \Theta(n \log(n))$ per query, which blows up over many queries.
- **Heap solution**: keep **two heaps**.
 - A **max-heap** holding the smallest $n/2$ elements (left).
 - A **min-heap** holding the largest $n/2$ elements (right).
 - The values needed for the median are always at the tops of the two heaps.

Algorithm

1. On each extracted element, decide which heap it belongs to:
 - If it is **larger than the top of the max-heap**, push into the **min-heap**; otherwise push into the **max-heap**.
 - If both heaps are empty, either heap works.
2. If the heap sizes differ by **more than one**, **rebalance**: pop the top of the larger heap and push it into the smaller one.
3. To compute the median:
 - Equal sizes (even number of elements) $\Rightarrow$ **average** of the two tops.
 - Different sizes (odd count) $\Rightarrow$ the **top of the larger heap**.

Walkthrough on stream 12, 55, 74, 35, 96, 52, ...

- **12**: both empty, put in left max-heap. Left {12}, Right {}.
- **55**: larger than left top $\Rightarrow$ right. Left {12}, Right {55}. Median = $(12 + 55)/2 = 33.5$.
- **74**: larger than left top $\Rightarrow$ right. Right {55, 74}. Median = 55 (right heap is larger).
- **35**: larger than left top $\Rightarrow$ right. Sizes now differ by more than one $\Rightarrow$ rebalance: move top of right (35) to left. Left {35, 12}, Right {55, 74}. Median = $(35 + 55)/2 = 45$.
- **96**: $\rightarrow$ right. Right {55, 74, 96}. Median = 55.
- **52**: $\rightarrow$ right, then rebalance (move 52 to left). Left {52, 12, 35}, Right {55, 74, 96}. Median = $(52 + 55)/2 = 53.5$.

10.7.3 Merging k Sorted Arrays

Given k sorted arrays of size n each, merge them into one sorted array.

- **Naive**: dump all nk elements into one array and sort $\Rightarrow \Theta(nk \log(nk))$.

Heap solution — $\Theta(nk \log(k))$

1. Create a **min-heap** and insert the **first element of each** of the k arrays.
 2. While the min-heap is not empty: pop the top, push it to the output array, and insert the **next element from the array that the popped element came from**.
- Heap size is k , so each push/pop is $\Theta(\log(k))$; nk elements are pushed and popped $\Rightarrow \Theta(nk \log(k))$.

Walkthrough

Arrays: [3,5,10,16], [2,4,8,12], [1,7,11,15], [6,9,13,14].

- Min-heap starts with {3, 2, 1, 6}. Pop **1** (array 3) $\rightarrow$ output [1], push 7.
- Pop **2** (array 2) $\rightarrow$ output [1,2], push 4.
- Pop **3** (array 1) $\rightarrow$ output [1,2,3], push 5.
- Pop **4** (array 2) $\rightarrow$ output [1,2,3,4], push 8. ... and so on.

Recursive solution — also $\Theta(nk \log(k))$

- Recursively merge the arrays **two at a time** until one remains. Merging two sorted arrays is $\Theta(n)$ (details in Chapter 13).
- Iteration 1: merge k arrays of size $n \Rightarrow \Theta(nk)$ work. Iteration 2: merge $k/2$ arrays of size $2n \Rightarrow (k/2) \times 2n = \Theta(nk)$ work. Every iteration costs $\Theta(nk)$.
- The number of arrays halves each iteration ($k \rightarrow 1$), so $\Theta(\log(k))$ iterations are needed.
- Total: $\Theta(\log(k)) \times \Theta(nk) = \Theta(nk \log(k))$ — asymptotically identical to the heap approach.